EXPERIMENT NO. 6

AIM:

To study and Implement Agile Software Project lifecycle using Jira with DevOps Pipeline Integration.

THEORY:

What is Jira?

Jira is an enterprise-grade project management and issue-tracking software developed by Atlassian. Originally designed as a bug tracking tool for software teams, Jira has evolved into the industry-standard platform for managing projects using Agile frameworks like Scrum and Kanban. It serves as a centralized hub where cross-functional teams can plan strategies, track real-time progress, manage software releases, and maintain high visibility across complex workflows.

Extended Theory of Agile & Scrum Principles in Jira

1. The Agile Ecosystem & Scrum Framework Agile is an iterative project management approach focused on continuous feedback, flexibility, and small incremental deliverables rather than rigid long-term plans. Scrum is a specific implementation of Agile that divides work into fixed-length iterations called Sprints (typically 1 to 4 weeks).

In Scrum, work is prioritized in a global repository known as the Backlog. During Sprint Planning, items are pulled from the backlog into a Sprint container based on team capacity and project goals. Once initiated, the active sprint is managed visually using boards to enforce accountability and maintain continuous workflow.

2. Hierarchical Work Breakdown Structure (WBS) To manage large projects without losing alignment, Jira uses a top-down structural hierarchy:

- Roadmap (Top-Level Strategy): High-level visual timeline spanning months or quarters, mapping out overall business objectives and long-term milestones.
- Epics (Strategic Initiatives): Large bodies of work that encompass broad functionalities (e.g., *Improve Customer Service* or *Increase Product Page Conversions*). Managed primary by the Product Owner, Epics take multiple sprints to complete and are split into smaller items.
- User Stories (Functional Value Units): Requirement specifications written from the end-user's perspective (e.g., *"Provide live chat as a feature"*). Stories define *what* needs to be built to deliver tangible customer value.

- Tasks & Sub-tasks (Technical Execution Units): Discrete technical actions assigned directly to developers (e.g., *"Add WhatsApp chat to our website"*). Tasks detail *how* a story will be technically implemented.

3. Dependency Mapping & Blocker Management In complex software architecture, issues rarely exist in isolation. Jira uses explicit Issue Linking to model real-world technical dependencies:

- Blocker Relationships: By linking tasks to stories using relationships like *"is blocked by"*, teams visually declare that a feature cannot proceed until prerequisite foundational tasks are finished.
- Risk Mitigation: Identifying dependencies upfront prevents bottlenecking during sprint execution, ensuring developer teams work on actionable items rather than waiting on incomplete infrastructure.

4. Active Boards & Custom Workflow Design Active Sprint boards translate backend issue records into visual Kanban-style cards across linear status columns:

- Standard States: Default columns include *To Do* (queued), *In Progress* (actively worked on), and *Done* (completed and verified).
- Custom Columns (e.g., Review): Projects often require intermediate quality checks. Adding custom stages like Review between *In Progress* and *Done* introduces a dedicated Quality Assurance (QA) and code-review gate, preventing unverified code from reaching production.

5. Reporting, Gantt Timelines & Ecosystem Integrations

- Timeline (Gantt) Views: Dynamically render Epics and child tasks over chronological axes, auto-drawing dependency vectors to pinpoint critical path delays.
- List Views: Provide spreadsheet-style interfaces for rapid bulk editing, metadata filtering, and audit checks.
- Confluence & Third-Party Integrations: Connect native documentation spaces for sprint retrospectives and link external tracking tools (e.g., Slack, TimeCamp) to aggregate project telemetry into customized Jira Dashboards.

In software project management and Agile frameworks (such as Scrum and Kanban), Push and Pull describe how work items move through a development workflow:

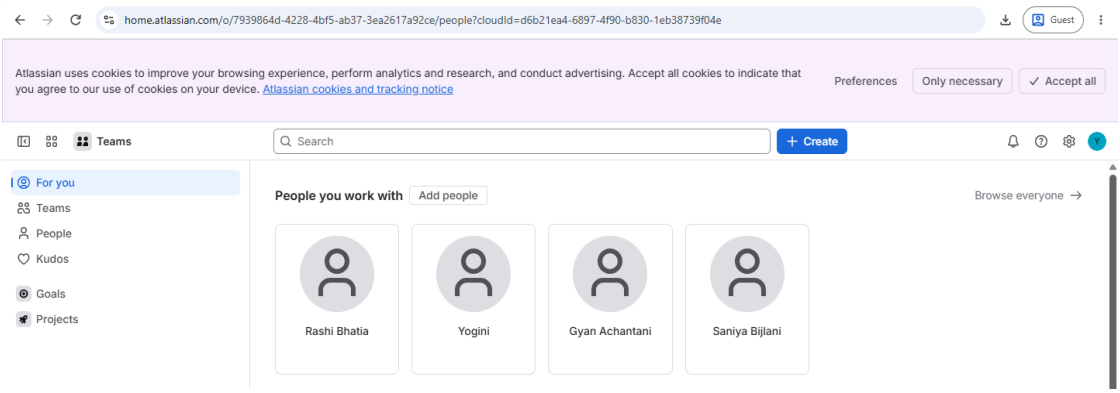
- Push System: Work is assigned or forced onto team members based on a predetermined schedule, regardless of their immediate capacity or current bottlenecks.
- Pull System: Team members autonomously select ("pull") tasks from a prioritized backlog into active workflow states (e.g., *To Do* → *In Progress*) only when they have explicit capacity to take on new work. This prevents work-in-progress (WIP) overburdening and improves delivery flow.

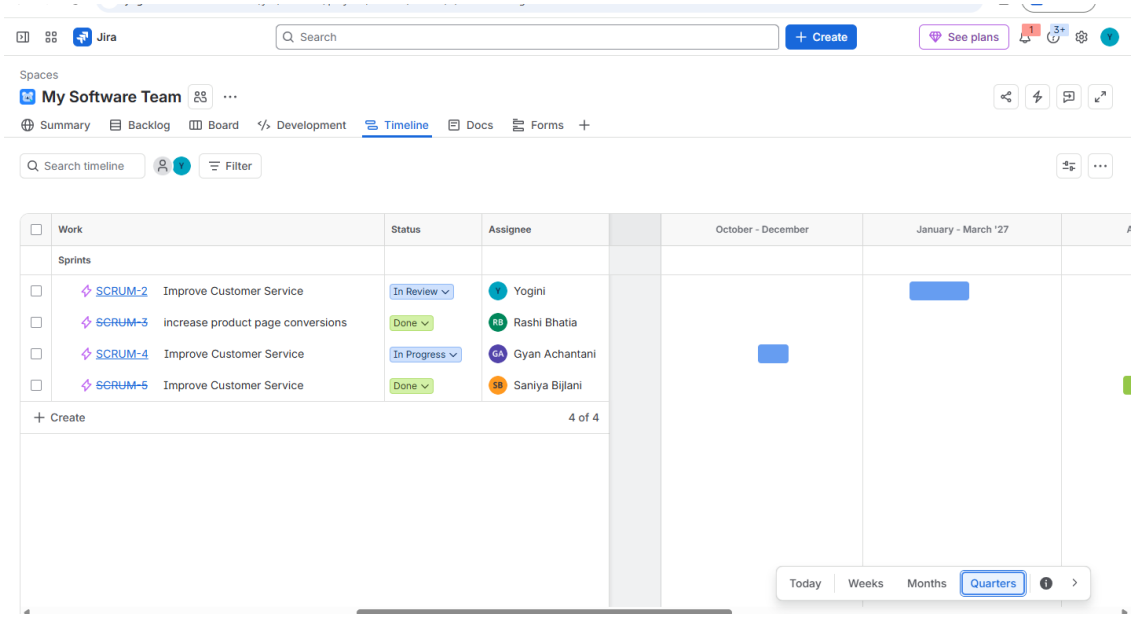
In Agile software development, the Push and Pull models define how work moves through a project lifecycle, balancing workload allocation with team capacity. A push system relies on assigning tasks to developers based on target deadlines or pre-scheduled plans, whereas a pull system empowers team members to actively pull prioritized user stories from the backlog into active workflow stages—such as moving issues from *To Do* to *In Progress*—only when they have the immediate capacity to work on them. Integrating these concepts into tools like Jira and version control platforms like Git harmonizes task management with technical implementation. When developers execute a git push with smart commit keys (such as referencing issue SWD-2), code updates are automatically pushed to the remote repository and synced back into Jira's development tracking. Conversely, opening a Git Pull Request allows reviewers to inspect changes while automatically pulling the corresponding Jira issue into custom board columns like *Review* or *Done*. In conclusion, combining pull-based task allocation with automated push/pull code integrations streamlines visibility across active boards and timeline views, preventing team burnout while maintaining full traceability from task assignment through final code delivery.

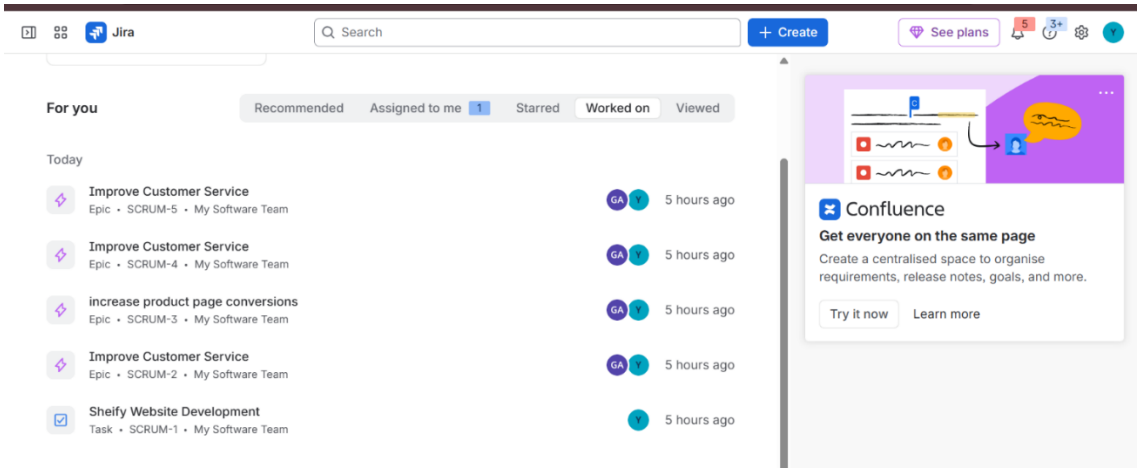
Key Technical Integration Points

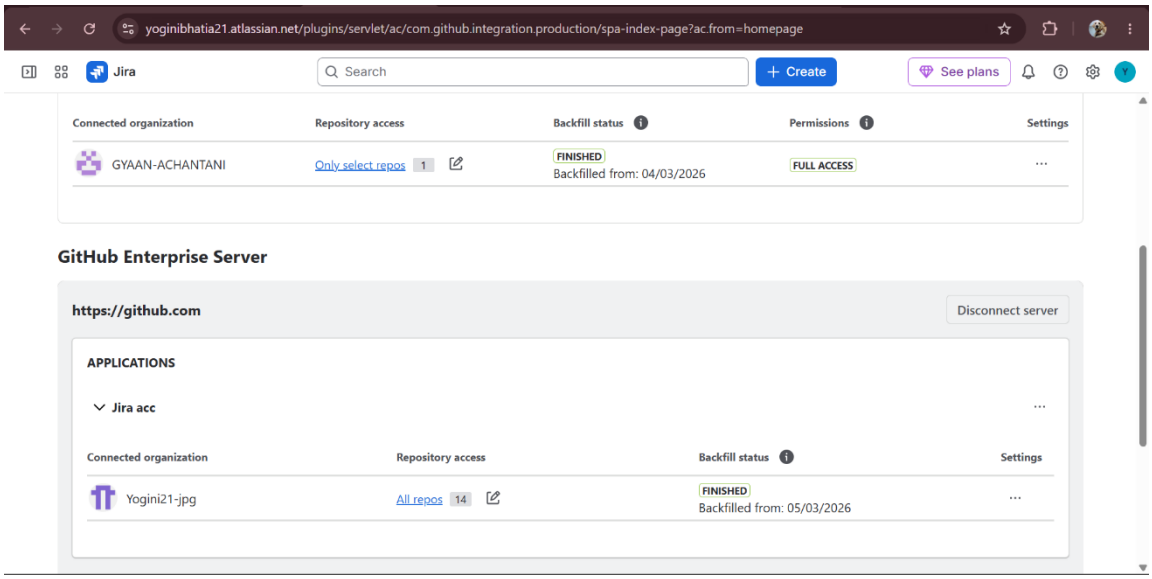
- **Smart Commits:** Developers include the Jira issue key (e.g., git commit -m "SWD-2: Add WhatsApp chat feature") during a git push.
- **Automated Sync:** Pushed commits and open Pull Requests automatically update Jira issue fields, record logged work, or transition tasks across custom board columns (such as *To Do* → *In Progress* → *Review* → *Done*).

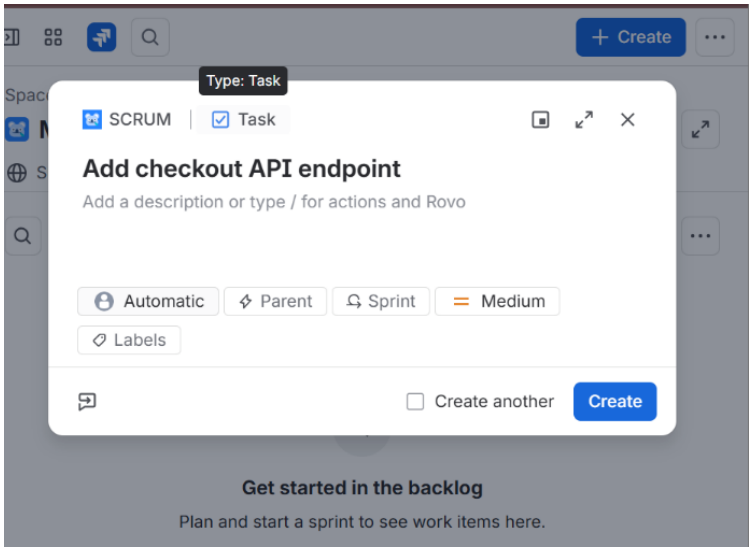
IMPLEMENTATION:

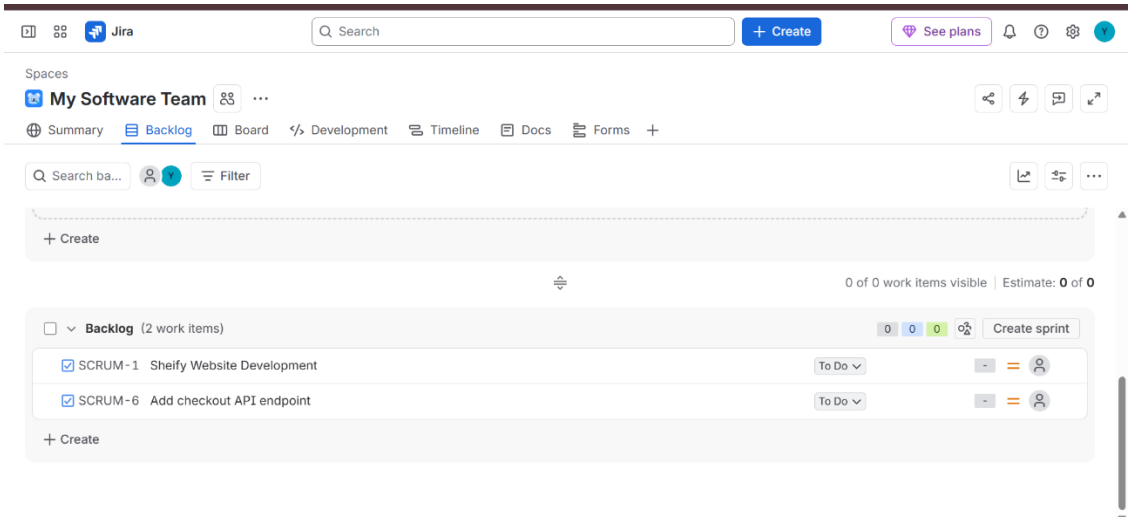



Jira
Search
Create
See plans
Spaces
My Software Team
Summary
Backlog
Board
Development
Timeline
Docs
Forms
Search timeline
Filter
Work
Status
Assignee
October - December
January - March '27
Sprints
SCRUM-2 Improve Customer Service
In Review
Yogini
SCRUM-3 increase product page conversions
Done
Rashi Bhatia
SCRUM-4 Improve Customer Service
In Progress
Gyan Achantani
SCRUM-5 Improve Customer Service
Done
Saniya Bijlani
Create
4 of 4
Today
Weeks
Months
Quarters


Jira
Search
Create
See plans
For you
Recommended
Assigned to me
Starred
Worked on
Viewed
Today
Improve Customer Service
Epic • SCRUM-5 • My Software Team
5 hours ago
Improve Customer Service
Epic • SCRUM-4 • My Software Team
5 hours ago
increase product page conversions
Epic • SCRUM-3 • My Software Team
5 hours ago
Improve Customer Service
Epic • SCRUM-2 • My Software Team
5 hours ago
Sheify Website Development
Task • SCRUM-1 • My Software Team
5 hours ago
Confluence
Get everyone on the same page
Create a centralised space to organise requirements, release notes, goals, and more.
Try it now
Learn more


yoginibhatia21.atlassian.net/plugins/servlet/ac/com.github.integration.production/spa-index-page?ac.from=homepage
Jira
Search
Create
See plans
Connected organization
Repository access
Backfill status
Permissions
Settings
GYAAN-ACHANTANI
Only select repos 1
FINISHED
Backfilled from: 04/03/2026
FULL ACCESS
GitHub Enterprise Server
https://github.com
Disconnect server
APPLICATIONS
Jira acc
Connected organization
Repository access
Backfill status
Settings
Yogini21-jpg
All repos 14
FINISHED
Backfilled from: 05/03/2026






```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Loading personal and system profiles took 3008ms.
(base) PS C:\Users\yogini bhatia> git checkout main
error: pathspec 'main' did not match any file(s) known to git
(base) PS C:\Users\yogini bhatia> git pull origin main
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
(base) PS C:\Users\yogini bhatia>
```

```
(base) PS C:\Users\yogini bhatia> git pull origin main
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
(base) PS C:\Users\yogini bhatia> cd ecommerce-app
(base) PS C:\Users\yogini bhatia\ecommerce-app> git init
Reinitialized existing Git repository in C:/Users/yogini bhatia/ecommerce-app/.git/
(base) PS C:\Users\yogini bhatia\ecommerce-app> git remote add origin https://github.com/Yogini21-jpg/ecommerce-
error: remote origin already exists.
(base) PS C:\Users\yogini bhatia\ecommerce-app> git branch -M main
(base) PS C:\Users\yogini bhatia\ecommerce-app> git add .
(base) PS C:\Users\yogini bhatia\ecommerce-app> git commit -m "Initial store code commit"
On branch main
Your branch is up to date with 'origin/main'.

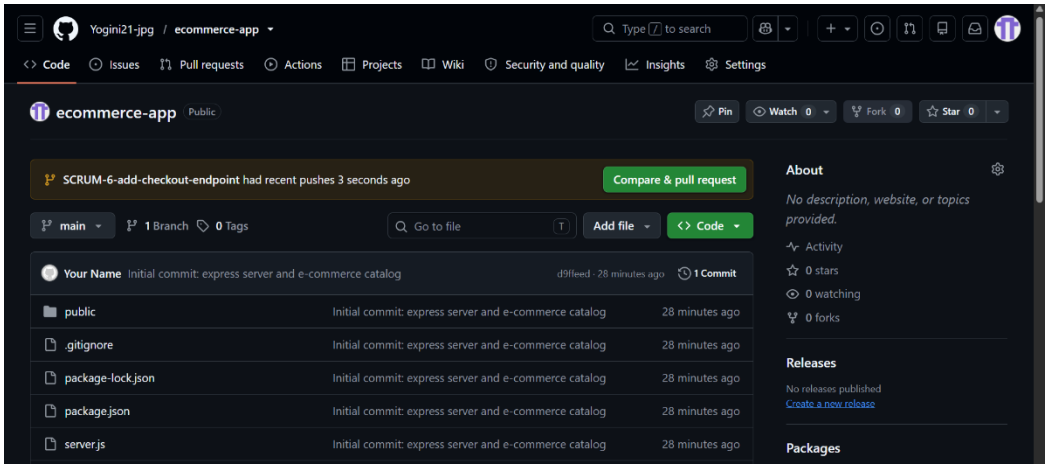
nothing to commit, working tree clean
(base) PS C:\Users\yogini bhatia\ecommerce-app> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
(base) PS C:\Users\yogini bhatia\ecommerce-app> git checkout -b SCRUM-6-add-checkout-endpoint
Switched to a new branch 'SCRUM-6-add-checkout-endpoint'
(base) PS C:\Users\yogini bhatia\ecommerce-app> git push -u origin SCRUM-6-add-checkout-endpoint
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'SCRUM-6-add-checkout-endpoint' on GitHub by visiting:
remote:      https://github.com/Yogini21-jpg/ecommerce-app/pull/new/SCRUM-6-add-checkout-endpoint
remote:
To https://github.com/Yogini21-jpg/ecommerce-app.git
 * [new branch]      SCRUM-6-add-checkout-endpoint -> SCRUM-6-add-checkout-endpoint
branch 'SCRUM-6-add-checkout-endpoint' set up to track 'origin/SCRUM-6-add-checkout-endpoint'.
(base) PS C:\Users\yogini bhatia\ecommerce-app>
```

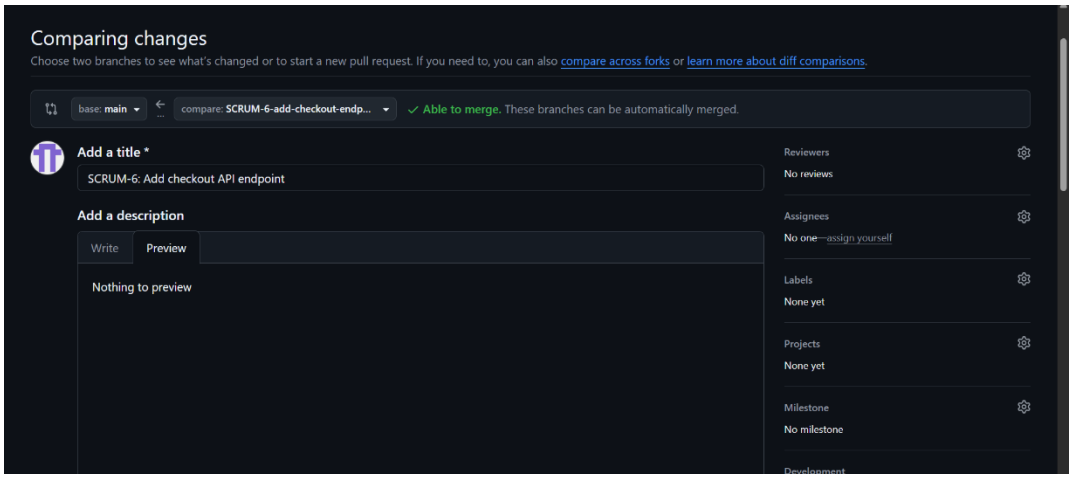
```
(base) PS C:\Users\yogini bhatia\ecommerce-app> notepad server.js
(base) PS C:\Users\yogini bhatia\ecommerce-app> git add server.js
(base) PS C:\Users\yogini bhatia\ecommerce-app> git commit -m "SCRUM-6: Add checkout API endpoint"
[SCRUM-6-add-checkout-endpoint 9da3b06] SCRUM-6: Add checkout API endpoint
 1 file changed, 7 insertions(+), 3 deletions(-)
(base) PS C:\Users\yogini bhatia\ecommerce-app> git push -u origin SCRUM-6-add-checkout-endpoint
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 445 bytes | 222.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Yogini21-jpg/ecommerce-app.git
   d9ffeed..9da3b06  SCRUM-6-add-checkout-endpoint -> SCRUM-6-add-checkout-endpoint
branch 'SCRUM-6-add-checkout-endpoint' set up to track 'origin/SCRUM-6-add-checkout-endpoint'.
(base) PS C:\Users\yogini bhatia\ecommerce-app>
```

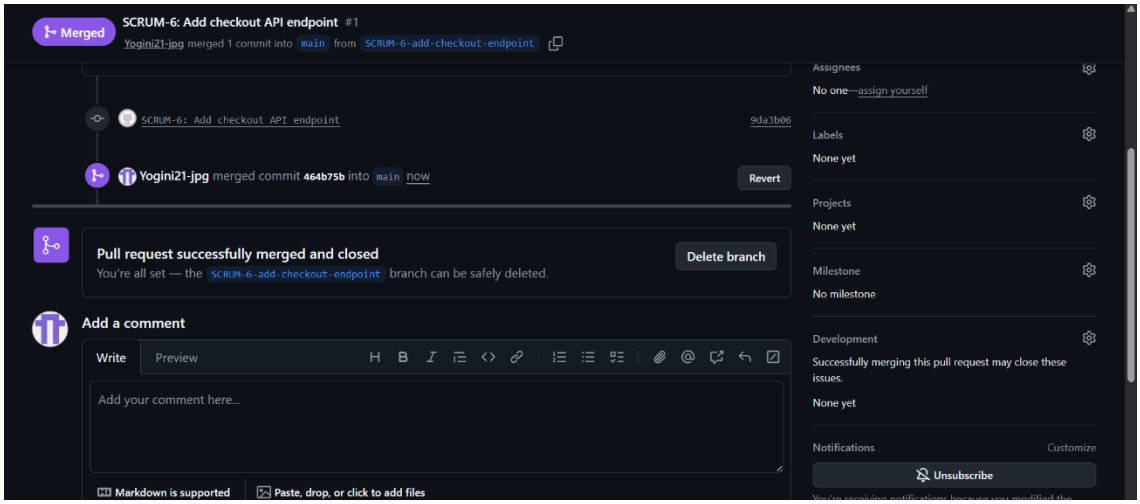
```
// API route
app.get('/api/products', (req, res) => {
    res.json(products);
});

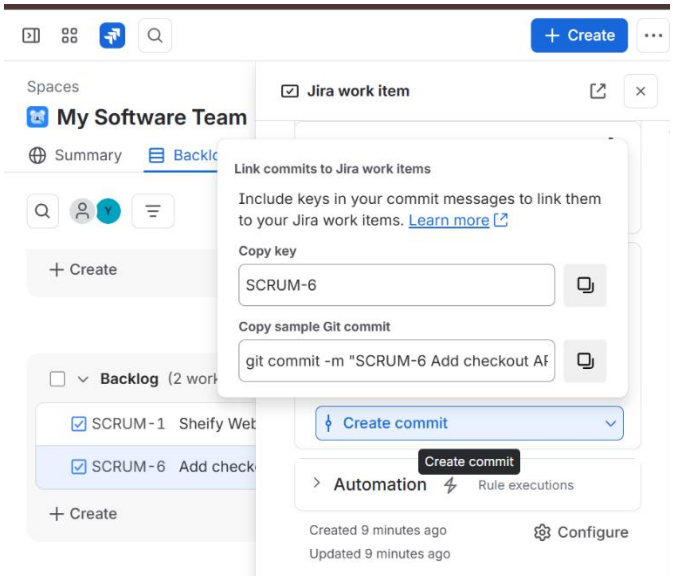
app.post('/api/checkout', (req, res) => {
    res.json({ message: 'Order processed successfully!' });
});

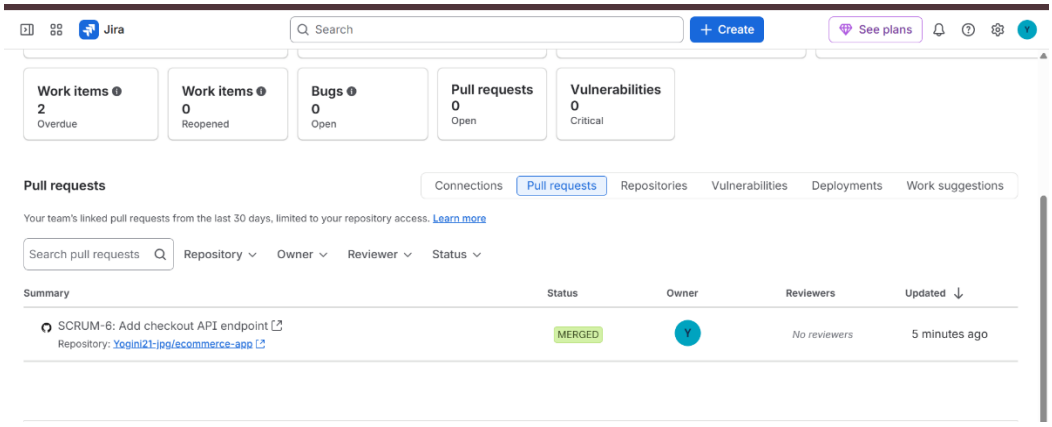
app.listen(PORT, () => {
    console.log(`Server is running at http://localhost:${PORT}`);
});
```

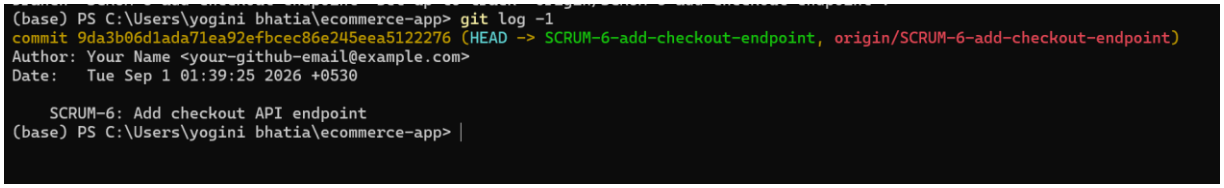

Yogini21-jpg / ecommerce-app
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
ecommerce-app
Public
Pin
Watch 0
Fork 0
Star 0
SCRUM-6-add-checkout-endpoint had recent pushes 3 seconds ago
Compare & pull request
main
1 Branch
0 Tags
Go to file
Add file
Code
Your Name Initial commit: express server and e-commerce catalog
d9ffeed · 28 minutes ago
1 Commit
public Initial commit: express server and e-commerce catalog 28 minutes ago
.gitignore Initial commit: express server and e-commerce catalog 28 minutes ago
package-lock.json Initial commit: express server and e-commerce catalog 28 minutes ago
package.json Initial commit: express server and e-commerce catalog 28 minutes ago
server.js Initial commit: express server and e-commerce catalog 28 minutes ago
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages

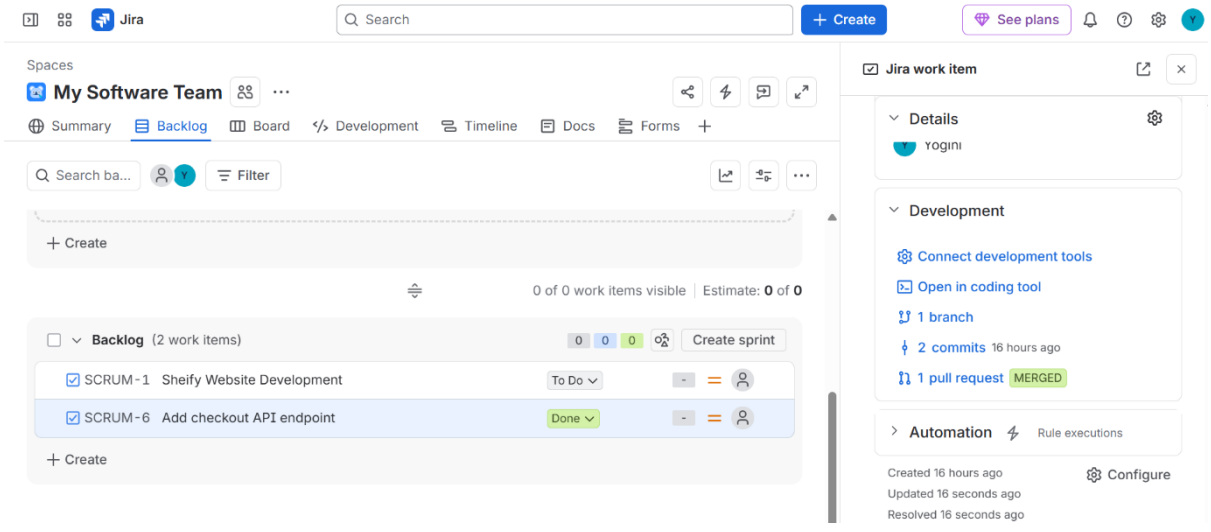

Comparing changes
Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.
base: main
compare: SCRUM-6-add-checkout-endp...
Able to merge. These branches can be automatically merged.
Add a title *
SCRUM-6: Add checkout API endpoint
Add a description
Write
Preview
Nothing to preview
Reviewers
No reviews
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone


Merged
SCRUM-6: Add checkout API endpoint #1
Yogini21-jpg merged 1 commit into main from SCRUM-6-add-checkout-endpoint
SCRUM-6: Add checkout API endpoint
9da3b06
Yogini21-jpg merged commit 464b75b into main now
Revert
Pull request successfully merged and closed
You're all set — the SCRUM-6-add-checkout-endpoint branch can be safely deleted.
Delete branch
Add a comment
Write
Preview
Add your comment here...
Markdown is supported
Paste, drop, or click to add files
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
None yet
Notifications
Customize
Unsubscribe


Create
Spaces
My Software Team
Summary
Backlog
Jira work item
Link commits to Jira work items
Include keys in your commit messages to link them to your Jira work items. Learn more
Copy key
SCRUM-6
Copy sample Git commit
git commit -m "SCRUM-6 Add checkout AP
Backlog (2 work
SCRUM-1 Sheify Web
SCRUM-6 Add check
Create commit
Create commit
Automation
Rule executions
Created 9 minutes ago
Updated 9 minutes ago
Configure


Jira
Search
Create
See plans
Work items 2 Overdue
Work items 0 Reopened
Bugs 0 Open
Pull requests 0 Open
Vulnerabilities 0 Critical
Pull requests
Connections
Pull requests
Repositories
Vulnerabilities
Deployments
Work suggestions
Your team's linked pull requests from the last 30 days, limited to your repository access. Learn more
Search pull requests
Repository
Owner
Reviewer
Status
Summary
Status
Owner
Reviewers
Updated
SCRUM-6: Add checkout API endpoint
Repository: Yogini21-jpg/ecommerce-app
MERGED
No reviewers
5 minutes ago


(base) PS C:\Users\yogini bhatia\ecommerce-app> git log -1
commit 9da3b06d1ada71ea92efbcec86e245eea5122276 (HEAD -> SCRUM-6-add-checkout-endpoint, origin/SCRUM-6-add-checkout-endpoint)
Author: Your Name <your-github-email@example.com>
Date: Tue Sep 1 01:39:25 2026 +0530
SCRUM-6: Add checkout API endpoint
(base) PS C:\Users\yogini bhatia\ecommerce-app>


Jira
Search
Create
See plans
Spaces
My Software Team
Summary
Backlog
Board
Development
Timeline
Docs
Forms
Search ba...
Filter
Create
0 of 0 work items visible
Estimate: 0 of 0
Backlog (2 work items)
Create sprint
SCRUM-1 Sheify Website Development
To Do
SCRUM-6 Add checkout API endpoint
Done
Create
Jira work item
Details
Yogini
Development
Connect development tools
Open in coding tool
1 branch
2 commits 16 hours ago
1 pull request MERGED
Automation
Rule executions
Created 16 hours ago
Updated 16 seconds ago
Resolved 16 seconds ago
Configure

CONCLUSION:

Following this structured Jira workflow ensures seamless Agile execution for the Sheify Website Development project. By organizing high-level strategies into Epics, detailing deliverables in the Backlog, enforcing explicit dependency blockages, and managing live work via custom board columns, teams maintain full transparency and accountability throughout the Sprint lifecycle.

Integrating Push and Pull mechanisms bridges task management in Jira with actual code execution in Git:

1. **Balanced Capacity:** Operating a Pull system ensures developers pull user stories based on realistic sprint capacity rather than being overloaded.
2. **Automated Traceability:** Coupling Git Pushes and Pull Requests directly with Jira issue keys eliminates manual tracking, ensuring transparent status visibility across boards and timeline views.